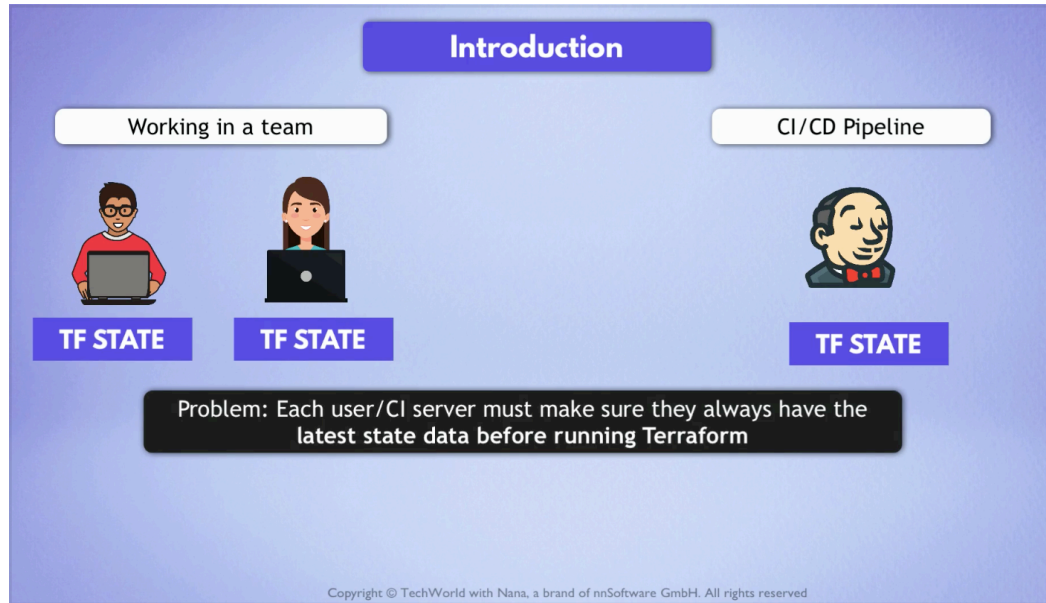


Module 12 - chapter 25

Remote State in Terraform

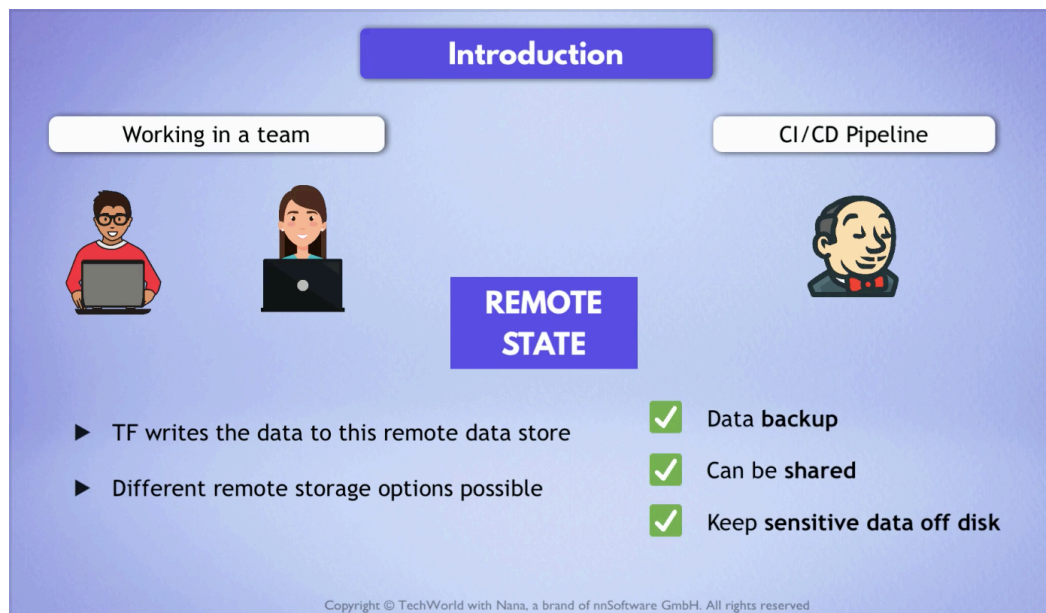
When working with terraform we create a **state**. And when a team works with terraform or we are using CI/CD we want that each user always have the latest state data before running terraform.



And in Module 12 - chapter 24 the state is stored in Jenkins Server, I can't execute terraform plan from my laptop as I don't have access to the state (unless I access the jenkins container -> workspace as I did to execute terraform destroy)

So how do we share the same terraform state file between different team members or different environments?

The way to do it and it is also best practice is to configure a **REMOTE STATE**



Configure Remote Storage

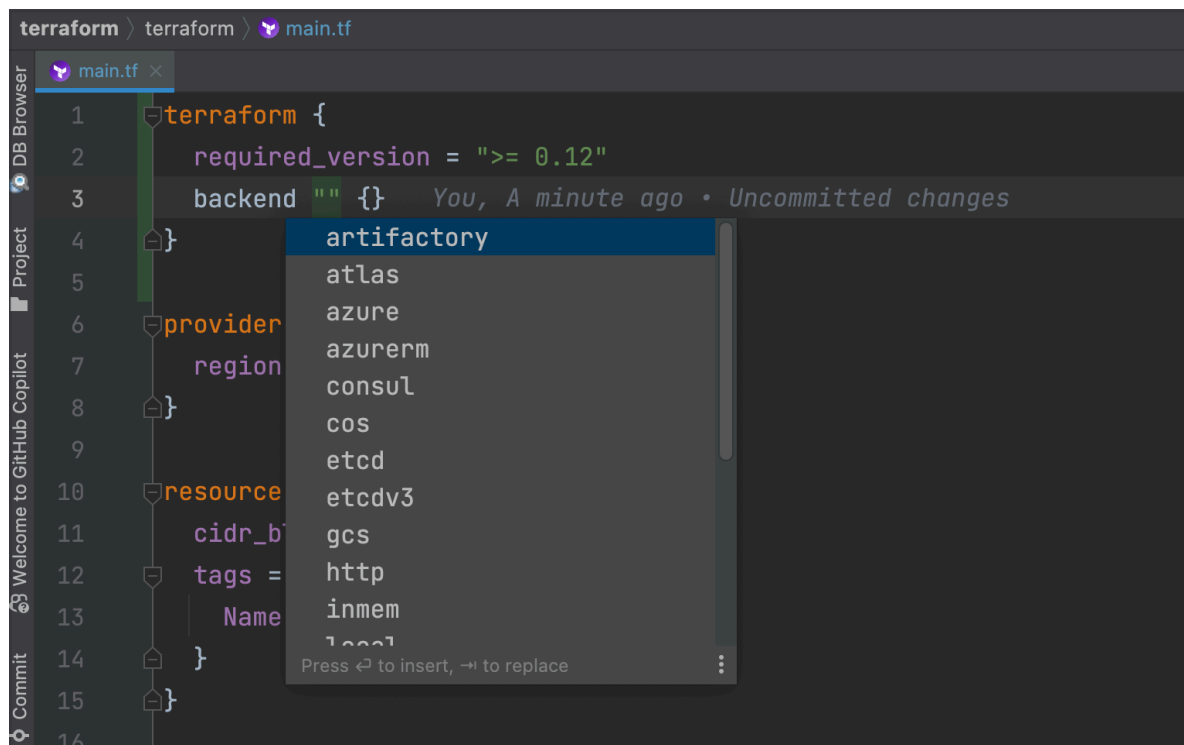
Configure Tf to store file remotely using the block `terraform { }`

```
terraform > terraform > main.tf
```

```
1 terraform {
2   |
3 }
4
```

`terraform { }` block is used to configure metadata and info about terraform itself. it configures Terraform itself, not our infrastructure resources.

Like the required version and backend which is the remote backend for tf



Backends

- Determines how state is loaded/stored
- Default: local storage

One of the remote backends is S3 bucket -> AWS simple storage, mostly used for storing files



```
1 terraform {
2     required_version = ">= 0.12"
3     backend "s3" {
4         bucket = "myapp-tf-s3-bucket"
5         key = "myapp/state.tfstate"
6     }
7 }
```

-> required_version = ">= 0.12"

A version constraint on the Terraform CLI. Anyone running this config must have Terraform 0.12 or newer, otherwise terraform init fails fast with a clear error. 0.12 was a notable cutoff because it introduced the modern HCL2 syntax — so this is essentially saying "don't run this with ancient Terraform."

-> backend "s3" { ... }

This configures remote state. By default Terraform stores its state file (terraform.tfstate) locally on disk. This tells it to store state in an AWS S3 bucket instead.

- bucket = "myapp-tf-s3-bucket" -> the S3 bucket where state lives.
- key = "myapp/state.tfstate" -> the path/object name of the state file inside that bucket.

Why use an S3 backend? Terraform state is the source of truth mapping your config to real cloud resources. Keeping it remote means:

- Your team shares one canonical state instead of each person having a local copy.
- State isn't lost if your laptop dies, and it's not committed to git (it can contain secrets).
- It enables collaboration on the same infrastructure.

Now we need to create the S3 bucket

Create AWS S3 Bucket



And let's create the bucket using the AWS UI:

I named it 'myapp-tf-s3-bucket' which is the name we are referencing in the terraform {} block

Amazon S3 > Buckets > Create bucket

Create bucket info

Buckets are containers for data stored in S3.

General configuration

AWS Region
Europe (London) eu-west-2

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#) ⓘ

☐ Global namespace
By default, S3 creates general purpose buckets in the global namespace.

☒ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name
For general purpose buckets in the account Regional namespace, the bucket name is composed of a bucket name prefix and a suffix. AWS adds the suffix to the bucket name prefix that you provide. This suffix is the account Regional namespace. The bucket name can't be edited after the bucket is created.

Bucket name prefix
myapp-tf-s3-bucket

Account Regional namespace suffix
-706976335940-eu-west-2-an

The bucket name prefix and the account Regional suffix combined must be between 3 and 63 characters long. The specified bucket name prefix must be unique within your account Regional namespace and follow the [bucket naming rules](#) ⓘ.

Full bucket name
myapp-tf-s3-bucket-706976335940-eu-west-2-an

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Another important config is '**Block Public Access settings for this bucket**'

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. [In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access.](#) These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#) ⓘ

☒ **Block all public access**
Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

- ☒ **Block public access to buckets and objects granted through new access control lists (ACLs)**
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.
- ☒ **Block public access to buckets and objects granted through any access control lists (ACLs)**
S3 will ignore all ACLs that grant public access to buckets and objects.
- ☒ **Block public access to buckets and objects granted through new public bucket or access point policies**
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.
- ☒ **Block public and cross-account access to buckets and objects through any public bucket or access point policies**
S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

So we keep it selected to block public access to our S3 bucket.

We have 'Bucket Versioning'

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of any object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

☒ Disable
 ☐ Enable

Which creates the versions of the files that we are storing in the bucket.

So every time a file content changes a new version is created -> so it is good practice to 'enable' it so we can track previous state when need it (not really a rollback, see questions at the end of notes)

The trade off is that we will have more files in the bucket.

We also have 'Default encryption' settings

- we keep the default 'Encryption type' -> SSE-S3
- Disable the 'Bucket key' -> it is for SSE-KMS not SSE-S3 so we won't use it in this demo

Default encryption [Info](#)

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [Info](#)

Secure your objects with two separate layers of encryption. For details on pricing, see [DSE-KMS pricing](#) on the [Storage](#) tab of the [Amazon S3 pricing page](#).

☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)
 ☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)
 ☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSE-KMS)

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSE-KMS. [Learn more](#)

☒ Disable
 ☐ Enable

Feature	SSE-S3	SSE-KMS
Manages keys	AWS manages everything automatically	You (optionally) manage via AWS KMS
Bucket Key option	Not applicable	Reduces KMS API calls (and cost)
Cost	Free	KMS has per-request charges

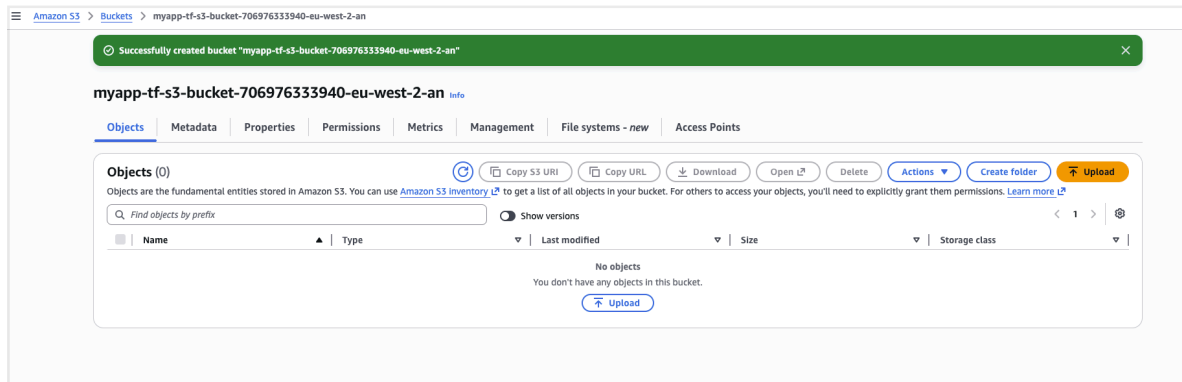
What the Bucket Key actually does:

When you use SSE-KMS, every S3 object operation (upload, download) makes a call to KMS to use your key — which costs money and has rate limits. Enabling the Bucket Key creates a short-lived key cached at the bucket level, reducing the number of calls to KMS (and therefore the cost and request overhead).

Why it's irrelevant for SSE-S3:

SSE-S3 never touches KMS at all. AWS manages the encryption keys internally using its own AES-256 implementation. There are no KMS API calls, no KMS costs, nothing. The Bucket Key toggle simply has no effect in this mode — disabling it is just being **explicit/clean**.

And I create the bucket



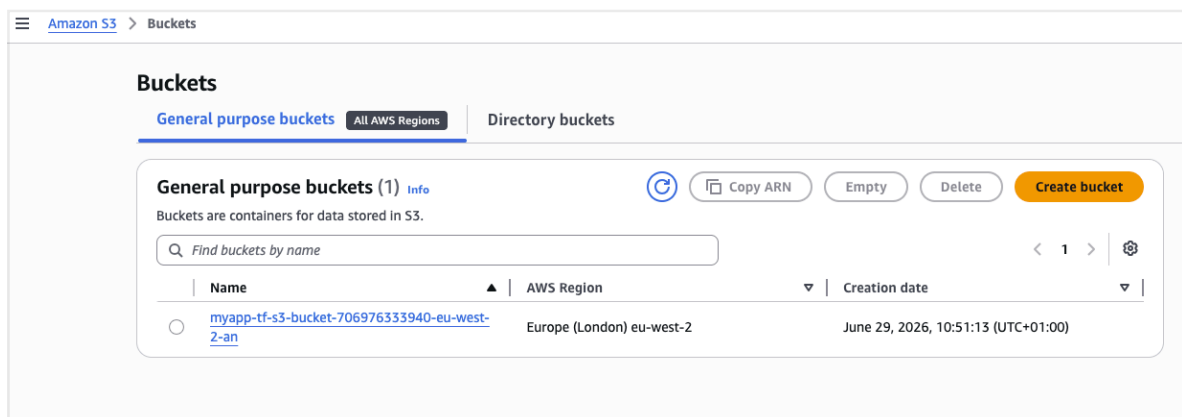
Currently our bucket is empty -> not uploaded files yet.

Note

make sure to update the full name of the s3 bucket in the terraform {} block

-> **myapp-tf-s3-bucket-706976333940-eu-west-2-an**

And we can see the region of our bucket



Which we can hardcoded in the terraform {} block as it won't change

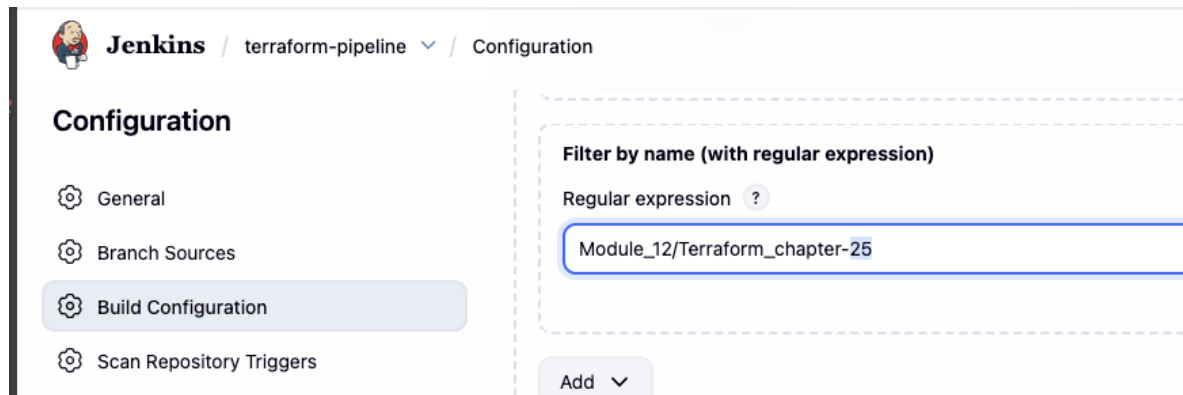
```
1 terraform {
2   required_version = ">= 0.12"
3   backend "s3" {
4     bucket = "myapp-tf-s3-bucket-706976333940-eu-west-2-an"
5     key = "myapp/state.tfstate"
6     region = "eu-west-2" // it doesn't have to be the same region as our TF infra, this is independent
7   }
8 }
```

And the region of the bucket can be any region as it is independent of the tf infra, this is just to store files.

So Tf will create the state file inside the bucket and it will keep updating that tf state file inside the bucket every time there is a change.

(At this point Nana recommends to destroy the TF infra from previous demo, but I have already done it)

next I update the branch in the pipeline config



So now I will push the changes and trigger the CI/CD

Execute Jenkins Pipeline

The S3 bucket was successfully configured

```
[Pipeline] sn
+ terraform init
[0m [1mInitializing provider plugins found in the configuration... [0m
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.52.0...
- Installed hashicorp/aws v6.52.0 (signed by HashiCorp)

[0m [1mInitializing the backend... [0m
[0m [32m
Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes. [0m

[31m] [0m [0m
[31m] [0m [0m [1m [31mError: [0m [0m [1mError refreshing state: Unable to access object "myapp/state.tf
operation error S3: HeadObject, https response error StatusCode: 403, RequestID: K36706SD4KW4A99K, Hos
TKfvU6QC14hsz+dD8uFqtZdU0k373K2lQxUG/0nq2FU46N6UZaVPa2YmYkxCkKAWfpGZDwcAm0ICKTipiHitqCFUX+GgDoqU, api
[31m] [0m [0m
[31m] [0m [0m [0m
```

But the pipeline fails, because my 'jenkins' user doesn't have permissions for the s3 bucket.



AI:

the backend is now connecting to the right bucket. This new error is a permissions problem: Terraform can reach the bucket but the IAM role/user Jenkins is using doesn't have permission to read/write objects in it.

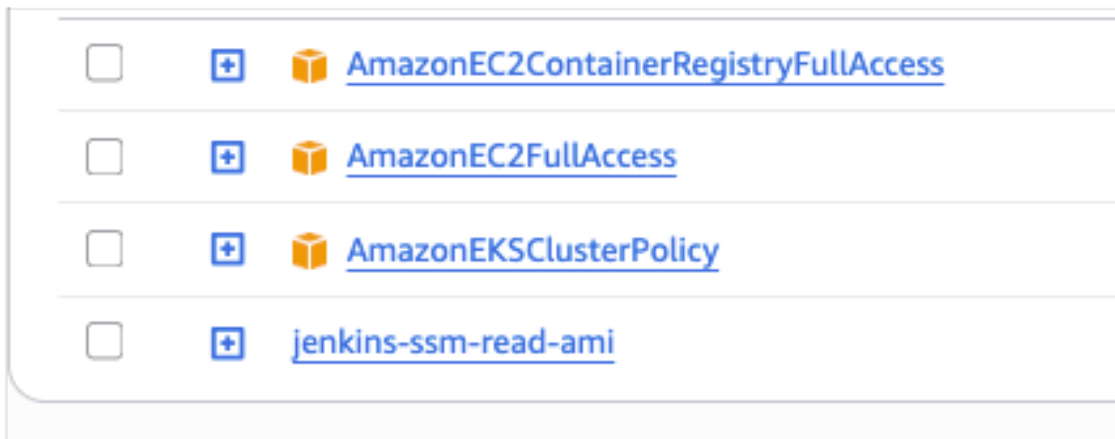
The HeadObject 403 means Jenkins' AWS credentials lack s3:GetObject (and likely other S3 permissions) on that bucket.

What needs to be in the IAM policy for the Jenkins role:

```
{
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:PutObject",
    "s3:DeleteObject",
    "s3:ListBucket"
  ],
  "Resource": [
    "arn:aws:s3:::myapp-tf-s3-bucket-706976333940-eu-west-2-an",
    "arn:aws:s3:::myapp-tf-s3-bucket-706976333940-eu-west-2-an/*"
  ]
}
```

Note the two resource entries — one for the bucket itself (ListBucket) and one for objects inside it (/).

These are my current permissions:



Your user has EC2, ECR, EKS, and SSM permissions but nothing for S3.

You need to attach **AmazonS3FullAccess** (or a scoped policy like the one I showed above) to this IAM user.

Step 1
 Add permissions
 Step 2
 Review

Add permissions

Add user to an existing group or create a new one. Using groups is a best-practice way to manage user's permissions by job functions. [Learn more](#)

Permissions options

☐ Add user to group
 Add user to an existing group, or create a new group. We recommend using groups to manage user permissions by job function.

☐ Copy permissions
 Copy all group memberships, attached managed policies, inline policies, and any existing permissions boundaries from an existing user.

☒ Attach policies directly
 Attach a managed policy directly to a user. As a best practice, we recommend attaching policies to a group instead. Then, add the user to the appropriate group.

Permissions policies (1/1508)

Search: amazons3fu Filter by Type: All types 1 match

Policy name	Type	Attached entities
☒ AmazonS3FullAccess	AWS managed	0

Cancel Next

done:

Permissions policies (5)

Permissions are defined by policies attached to the user directly or through groups.

Search: Filter All t

Policy name	Type
☐ AmazonEC2ContainerRegistryFullAccess	AWS managed
☐ AmazonEC2FullAccess	AWS managed
☐ AmazonEKSClusterPolicy	AWS managed
☐ AmazonS3FullAccess	AWS managed
☐ jenkins-ssm-read-ami	Customer inline

I re-build the jenkins job and it works:

- The backend is initialised
- And jenkins executes -> terraform apply -auto-approve

```
[Pipeline] sh
+ terraform init
[0m [1mInitializing provider plugins found in the configuration... [0m
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.52.0...
- Installed hashicorp/aws v6.52.0 (signed by HashiCorp)

[0m [1mInitializing the backend... [0m

Terraform has created a lock file [1m.terraform.lock.hcl [0m to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future. [0m

[0m [1m [32mTerraform has been successfully initialized! [0m [32m [0m
[0m [32m
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary. [0m
[Pipeline] sh
+ terraform apply -auto-approve
[0m [1mdata.aws_ssm_parameter.latest-amazon-linux-image: Reading... [0m [0m
[0m [1mdata.aws_ssm_parameter.latest-amazon-linux-image: Read complete after 0s [id=/aws/service/ami-amazon-

Terraform used the selected providers to generate the following execution
plan. Resource actions are indicated with the following symbols:
[32m+ [0m create [0m

Terraform will perform the following actions:
```

If we had kept the infrastructure from the previous demo, we needed to migrate the state from jenkins workspace to the S3 bucket, so I would've had to ssh into my droplet and go inside jenkins docker and then the workspace folder to execute -> terraform init is executed there would have been a question prompted at us about migrating and we would have had to confirm the migration.

The flow would have been:

1. Exec into the Jenkins container:
docker exec -it <jenkins-container> bash
2. Navigate to the workspace where Terraform ran:
cd /var/jenkins_home/workspace/<your-job-name>
3. Run terraform init there — because that's where the local terraform.tfstate file lives. Terraform sees the existing local state, notices you've now configured an S3 backend, and prompts:
Do you want to copy existing state to the new backend?
3. You type yes and it migrates it to S3.

Why it has to be done there: terraform init looks for state in the current directory. If you ran it anywhere else there'd be no local state file to migrate from — it would just initialise fresh.

Why it's not relevant in your case: You're starting from scratch with no prior

state, so there's nothing to migrate. Terraform init just initialises the S3 backend cleanly, which is what you're doing now.

My pipeline worked and all the resources were created in AWS

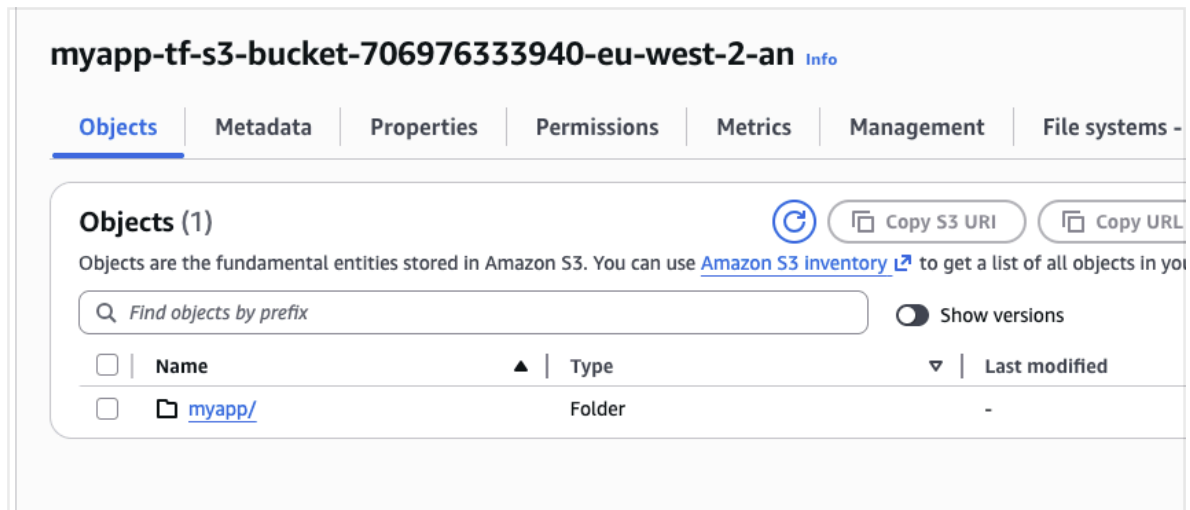
```
[0m [1maws_subnet.myapp-subnet-1: Creation complete after 15s [id=subnet-0e0e03100c4912204] [0m
[0m [1maws_default_route_table.main-rtb: Creation complete after 0s [id=rtb-0c2b48fd3d88f280d] [0m
[0m [1maws_default_security_group.default-sg: Creation complete after 2s [id=sg-018c72b5e050d2aa8] [0m
[0m [1maws_instance.myapp-server: Creating... [0m [0m
[0m [1maws_instance.myapp-server: Still creating... [00m10s elapsed] [0m [0m
[0m [1maws_instance.myapp-server: Still creating... [00m20s elapsed] [0m [0m
[0m [1maws_instance.myapp-server: Creation complete after 22s [id=i-059e7b8e046bbeaf5] [0m
[0m [1m [32m
Apply complete! Resources: 6 added, 0 changed, 0 destroyed. [0m
[0m [1m [32m
Outputs:

[mec2_public_ip = "18.170.224.121"
[Pipeline] sh
+ terraform output ec2_public_ip
Did you forget the `def` keyword? WorkflowScript seems to be setting a field named EC2_PUBLIC_IP
[Pipeline] }
```

And my app along with Postgres have been deployed

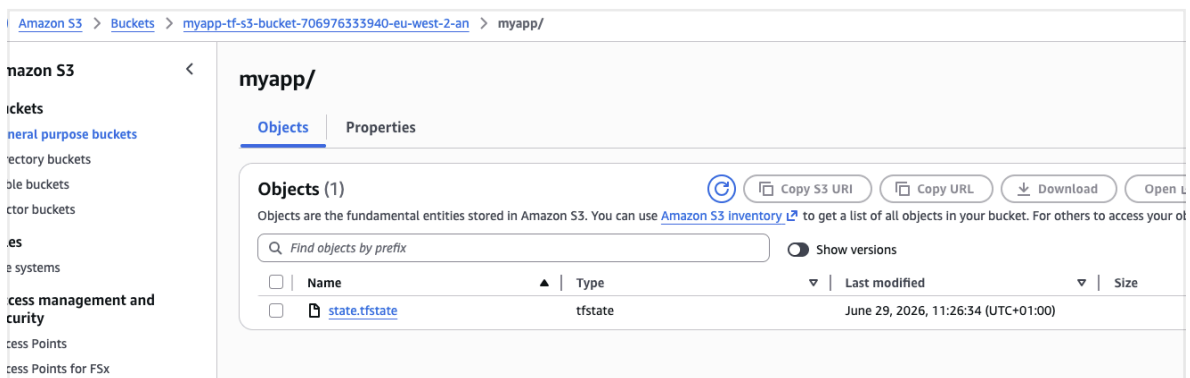
	Declarative: Checkout SCM	Declarative: Tool Install	build app	build image	provision server	deploy
Average stage times: (full run time: ~3min 20s)	1s	176ms	8s	16s	25s	28s
#4 Jun 29 11:25 No Changes	1s	118ms	8s	17s	53s	1min 54s
#3 Jun 29 11:15 1 commit	1s	142ms	7s	17s	16s failed	74ms failed
#2 Jun 29 11:06 1 commit	1s	198ms	9s	17s	16s failed	103ms failed
#1 Jun 29 11:02 No Changes	2s	248ms	8s	12s	14s failed	572ms failed

We can check the bucket

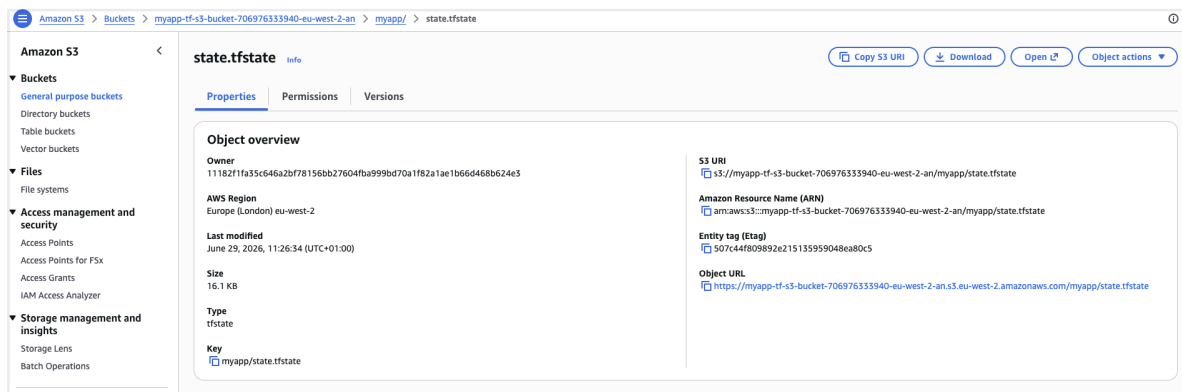


So we can see the folder we defined in the terraform {} block 🙌 myapp/

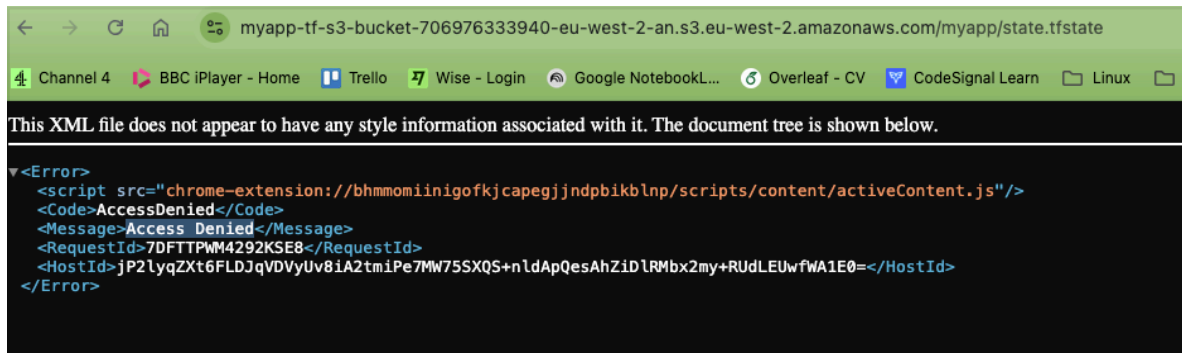
And inside the folder we have the state file



And this file has the current state of our infra.



And we have an URL but it is of no use because we blocked external access to the S3 bucket



We can access the file using AWS CLI or TF using AWS credentials.

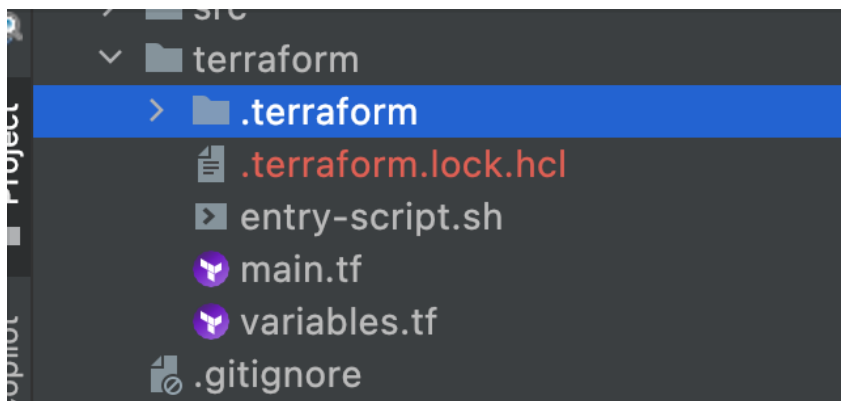
Now I go to IntelliJ and cd into the terraform folder and execute
-> terraform init

```
> cd terraform
> terraform init
Initializing provider plugins found in the configuration...terraform init
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.52.0...
- Installed hashicorp/aws v6.52.0 (signed by HashiCorp)

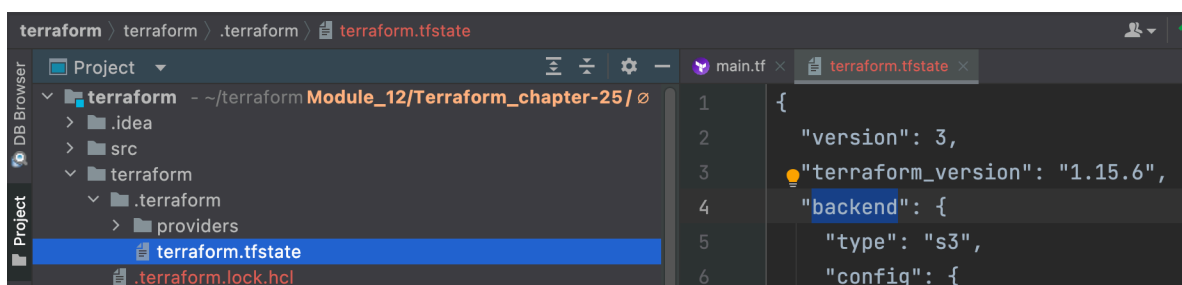
Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically
```

Now I have the folder '.terraform' and the file 'terraform.lock.hcl'



And I can see that the '.terraform' folder has a 'terraform.tfstate' file



And the 'terraform.tfstate' file only contains backend information.

Then I can access the share remote state file:

-> terraform state list

```
) terraform state list
data.aws_ssm_parameter.latest-amazon-linux-image
aws_default_route_table.main-rtb
aws_default_security_group.default-sg
aws_instance.myapp-server
aws_internet_gateway.myapp-igw
aws_subnet.myapp-subnet-1
aws_vpc.myapp-vpc
```

Here's what's happening:

terraform init locally just configures the connection to the S3 backend — it doesn't pull the state down to your laptop. It creates a local `.terraform/` folder with backend configuration metadata, and a minimal `terraform.tfstate` that just says "my backend is S3 over there." No actual infrastructure state lives locally.

`terraform state list` then talks directly to S3 to fetch and display the state. Your laptop never stores the real state — every state command goes live to the bucket.

Think of it like a database client: `terraform init` is like configuring the connection string, and every subsequent command (`state list`, `plan`, `apply`) queries the remote database directly. The state never truly "lives" on your laptop.

This is exactly why remote backends are useful for teams — everyone's local Terraform points at the same S3 bucket, so you all see the same state.

Questions

Q:

what happens if I run `terraform apply`? it will change the s3 bucket file?

A:

Yes. Every `terraform apply` rewrites the state file in S3 to reflect the new real-world infrastructure.

The flow is:

1. Read — Terraform fetches current state from S3
2. Plan — compares state against your config to work out what needs to change
3. Apply — makes the changes in AWS
4. Write — updates `myapp/state.tfstate` in S3 with the new state

So the S3 file is always the live record of what Terraform believes exists in your

infrastructure. Every apply overwrites it with the latest snapshot.

This is also why state locking matters — if two people ran terraform apply at the same time, both would read the same state, make changes, then both try to write back, and one would clobber the other's update. That's the DynamoDB locking table I mentioned earlier — it prevents concurrent writes.

Q:

how can we prevent doing this? should it be that we have a dev environment and we create a branch where we get a copy of the current state and do changes locally and then when we are happy we create a PR and once it is approved and merged then a new s3 bucket file is created with a new version?

A:

You're thinking in the right direction. A few approaches:

Terraform Workspaces — the most direct answer to your idea. Each workspace gets its own state file in S3:

myapp/env:/dev/state.tfstate

myapp/env:/staging/state.tfstate

myapp/env:/prod/state.tfstate

Each environment is isolated. Changes in dev don't touch prod state.

Git branch + CI/CD workflow — which is close to what you described:

- Branch → PR → review → merge to main → Jenkins runs terraform apply on main only
- terraform plan runs on PRs so reviewers can see what would change before approving
- Only the merge to main triggers an actual apply (and therefore an S3 state update)
- This is the standard pattern in production teams

S3 Versioning — not prevention, but safety net. Enable versioning on your state bucket and S3 keeps every previous version of state.tfstate. If an apply goes wrong you can roll back to a previous state file. Worth enabling regardless.

In practice these are combined:

- Workspaces for environment isolation
- Branch + PR workflow so apply only runs on approved, merged code
- S3 versioning as a safety net
- DynamoDB locking to prevent concurrent applies

Your instinct about "copy of state, make changes, PR, then write back" is essentially how the branch + plan-on-PR workflow works — plan is your preview without touching state, apply on merge is the commit.

Q:

and I can make a PR using only tf plan no need to use apply

A:

Exactly. The typical PR workflow is:

- On PR open/update → Jenkins runs terraform plan only — shows reviewers what would change, no state is touched
- On merge to main → Jenkins runs terraform apply — actually makes the changes and updates S3

So plan is your safe, read-only preview. It reads from S3 state but never writes back. You can run it as many times as you want with zero side effects.

This is why you often see Jenkinsfiles or GitHub Actions with two separate stages/jobs gated on the branch:

```
stage('Plan') {
  steps {
    sh 'terraform plan' // always runs
  }
}
stage('Apply') {
  when {
    branch 'main' // only on main
  }
  steps {
    sh 'terraform apply -auto-approve'
  }
}
```

```

stage('Plan') {
  steps {
    sh 'terraform plan' // always runs
  }
}
stage('Apply') {
  when {
    branch 'main' // only on main
  }
  steps {
    sh 'terraform apply -auto-approve'
  }
}

```

The reviewer looks at the plan output in the PR, approves if they're happy, and only then does the merge trigger the real apply.

Q:

and if now by mistake I run terraform apply from my laptop (I have the versioning enabled) will a new state file be created?

A:

Yes. If you run terraform apply from your laptop it will:

1. Read the current state from S3
2. Make real infrastructure changes in AWS
3. Overwrite myapp/state.tfstate in S3 with the new state

But because you have versioning enabled, S3 doesn't delete the old file — it keeps it as a previous version. So you'd have:

myapp/state.tfstate (current — post your accidental apply)
 myapp/state.tfstate (version 2 — previous)
 myapp/state.tfstate (version 1 — original)

You can restore a previous version from the S3 console if needed.

However — restoring the state file doesn't undo the actual AWS infrastructure changes. If your apply created or deleted real resources, those changes are still there. State versioning is a safety net for the file, **not a rollback** of real infrastructure. **!!**

The proper guard against accidental local applies is to restrict IAM permissions so only the Jenkins role can call terraform apply — your personal IAM user would only have read access to S3 state and terraform plan level permissions.

That's the production pattern.

Q:

so from my laptop when I run terraform apply it is using the AWS user that I configured with AWS configure ?

Yes exactly. Whatever credentials you configured with aws configure on your laptop are what Terraform uses — so if that user has full permissions, it can apply and make real changes.

This is why in production the IAM setup is usually:

- Your personal user — read-only S3, terraform plan permissions only. Can inspect but not change.
- Jenkins role — full permissions to apply. Only CI/CD can make real changes.

So even if you ran terraform apply from your laptop by mistake, it would fail with a 403 — IAM stops it before anything happens. The CI/CD pipeline is the only path to actually changing infrastructure.

In your current setup though, your user likely has enough permissions (now that you added S3FullAccess), so there's nothing stopping a local apply — worth keeping in mind while you're learning.

Me:

In AWS configure I am using the credentials for the AWS 'admin' user which has full permissions. So I need to be careful !!

Using the AWS CLI command I can verify the user in my 'AWS configure'
-> aws sts get-caller-identity

```
{
  "UserId": "AIDA2JGYSYB2APKYYUP6D",
  "Account": "706976333940",
  "Arn": "arn:aws:iam::706976333940:user/admin"
}
(END)
```

Q:

so to be safe I should restrict the user that I got for AWS configure to only read s3 and if I execute terraform apply from my laptop it won't do anything

A:

A safe personal user policy would be:

- S3 — read only on the state bucket (GetObject, ListBucket — no PutObject)

- EC2, EKS, ECR etc. — read only (Describe*, List*, Get*)

That way:

- terraform plan works fine — reads state from S3, reads real infra, shows you the diff
- terraform apply fails with 403 — can't write back to S3 state, can't create/modify/delete real resources

The Jenkins role keeps full permissions so CI/CD can still apply.

This is a clean separation — your laptop is read and preview only, Jenkins is the only write path. Even if you accidentally ran terraform apply locally nothing would happen.